

Module 1 — Core Mastery

Lesson 1.1

The CLI surface

Know the modes, flags, and lifecycle that govern every Claude Code session.

Mastering Claude Code — An E-Course for Power Users

Why it matters

Most users only know `claude` and `claude --resume`. There are roughly a dozen other knobs that change what Claude can do, who it asks before doing it, and how much it remembers. Knowing them turns ad-hoc usage into a workflow.

The modes

There are three operational modes you should understand:

Mode	How to enter	What it does
Interactive	<code>claude</code>	The default. Read–edit–write–run loop with prompts on risky tools.
Plan	<code>claude --permission-mode plan</code> , or Shift+Tab in session	Read-only; Claude can investigate and propose, but cannot edit or run side-effects.
Headless	<code>claude -p "prompt"</code>	Single-shot, non-interactive. Designed for pipes, scripts, CI.

Headless mode is critical for automation (covered in Module 7).

Sessions and resumption

Each interactive run is a session. Sessions are saved automatically and identified by ID. The flags you care about:

```
claude          # new session
claude --continue # resume the most recent session in this dir
claude --resume  # interactive picker of recent sessions
claude --resume <id> # resume a specific session by ID
```

A session retains conversation history, the working directory context, and any in-flight plan. Use `/clear` to wipe the in-session context without ending the session.

Permission modes (in more depth)

Permissions decide which tools run with your approval vs. silently:

- `default` — prompts for tool calls not on the allowlist
- `plan` — read-only, no tool calls with side effects
- `acceptEdits` — auto-accepts file edits but still asks for shell commands
- `bypassPermissions` (aka `--dangerously-skip-permissions`) — no prompts, anything goes

Set at launch:

```
claude --permission-mode acceptEdits
```

Or change mid-session with Shift+Tab. Set defaults per-project in `.claude/settings.json` (covered in 3.2).

Useful launch flags

```

claude -p "summarize this repo"           # headless single-shot
claude -p "." --output-format json         # machine-readable output
claude --model claude-opus-4-7            # pick a specific model
claude --add-dir ../other-repo            # expose another directory
claude --mcp-config ./mcp.json            # load a specific MCP config
claude --print-system-prompt              # debug: print the resolved system prompt

```

Run `claude --help` against your installed version — flags evolve.

The session lifecycle

```

launch ■■■ load CLAUDE.md, settings, hooks, MCP servers
  ▼
  user turn ■■■ assistant turn (text + tool calls)
    ▲                                     ■
    ■■■■■ compaction (if needed) ■■■
    ▼
    /exit ■■■ persist session for /resume

```

Key insight: every session boot reloads CLAUDE.md, settings.json, hooks, and MCP servers. You can edit those mid-session, but they take effect on the next session.

Try it

- 1 Start a session with `claude`. Run `/status` (if available in your version) or simply note the working directory.
- 2 Exit. Start a new session with `claude --continue` — confirm history is intact.
- 3 Launch with `--permission-mode plan` and try to ask Claude to edit a file. Note that it can read and propose but not write.
- 4 Run headless: `claude -p "list the top-level files here in one sentence"` and pipe the output: `claude -p "." --output-format json | jq .`

Gotchas

- `--continue` is **dir-scoped**: it resumes the most recent session whose working directory matches. Run from the right folder.
- **MCP servers reload only at boot**: editing the MCP config mid-session doesn't take effect until you re-launch.
- **Plan mode is sticky**: if you Shift+Tab into plan mode, remember to Shift+Tab back out. Otherwise edits will fail silently with "plan mode active" messages.

Further reading

- 1.4 Plan mode — deep dive on plan workflow
- Module 3 Customization — pin permissions and env per project

- Official CLI reference: <https://docs.claude.com/en/docs/claude-code/cli-reference>

Next

→ 1.2 Files, search, and the right tool for the job